



LLM - Large Language Models

Benjamin Piwowarski

Lesson 1: Introduction (master MIND)

<https://master-mind.isir.upmc.fr/llm/slides>

Press ? for help on keyboard shortcuts

[Slides in PDF](#)



Machine Learning &
Deep Learning for
Information Access

CHAT GPT

3

CHATGPT

3RD-YEAR



OUTLINE

1. Language models and Transformer Architectures
2. Generating and Training
3. Efficient LLMs: inference & architectures
4. Prompting and Evaluation
5. Information Retrieval & RAG
6. **Multi-***: Multi-Modal, Multi-Lingual LLMs and Tools
7. **Trust, transparency and reliability**: alignment & Steering, biases, hallucinations & safety, prompt injection, explainability and interpretability

MARKS

Final Exam (50%)

- One week before the exam: you will be given an article to read
- Questions on the paper ⊕ general questions on the course

Practical (50%)

- Project (code + report) on the themes seen during practical lessons

TO COMMUNICATE

Personal matters

`benjamin.piwowarski@cnrs.fr`

General questions

🔑 Mattermost (LLM)

- Introduction
- Tasks and applications
- Distributed Word Representation
- Neural Models for Natural Language Processing
 - Quick glance at "history" (CNN, RNN)
 - (Self) Attention Models: Transformer-based Architectures
- Training Transformers and LLMs (some hints)

TEXT AND APPLICATIONS

DIFFERENT SOURCES, DIFFERENT FORMS



ALWAYS COMPLEX STRUCTURE

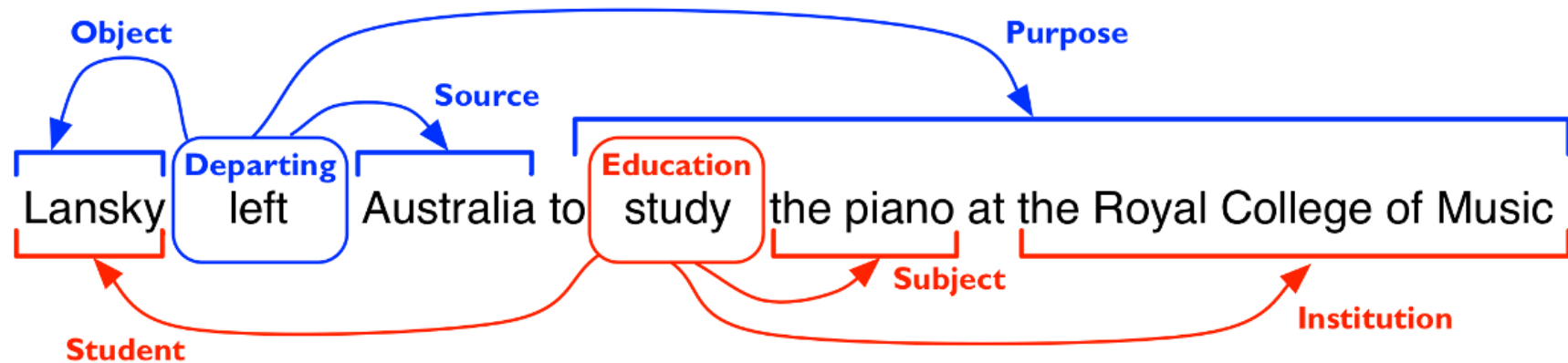
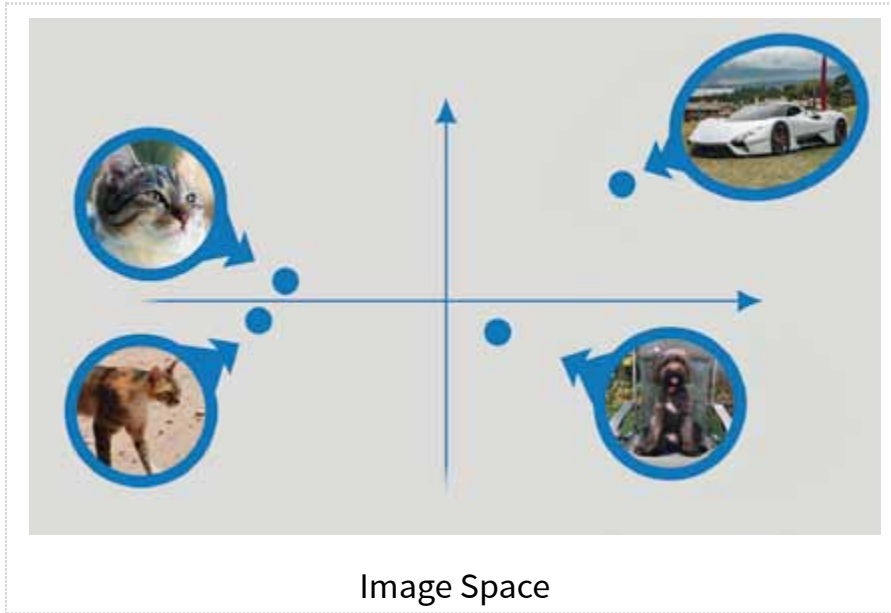


IMAGE = CONTINUOUS SPACE



TEXT = ARBITRARINESS OF THE SYMBOL

How to compare *Cat* ... *Dog* ?
No natural distance

Compositionality

The meaning of a complex expression is determined by the meanings of its constituent parts and the rules used to combine them

Examples

- **Word level:** "un" + "happy" → "unhappy"
- **Phrase level:** "red" + "car" → "red car"
- **Sentence level:** "The cat" + "sat on the mat" → "The cat sat on the mat"

Challenge

Understanding language requires understanding both individual units AND their composition



Polysemy and Ambiguity

The same word or sentence can have different meanings depending on context



Word level

- **"bank"**: financial institution vs. river bank
- **"bat"**: flying mammal vs. sports equipment



Sentence level

- **"I saw her duck"**
 - witnessed her lower her head
 - observed her pet waterfowl
- **"The chicken is ready to eat"**
 - the chicken is cooked and prepared for consumption
 - the chicken is hungry and wants to eat



Implication for NLP

Word and sentence representations must be **contextual** to capture meaning accurately

TEXT AND APPLICATIONS ► A LOT OF TEXT-RELATED TASKS

- Indexing and Information Retrieval
- Thematic classification / sentiments
- Translation
- Information extraction
- Question answering
- ...

Two main families of models: classification and generation

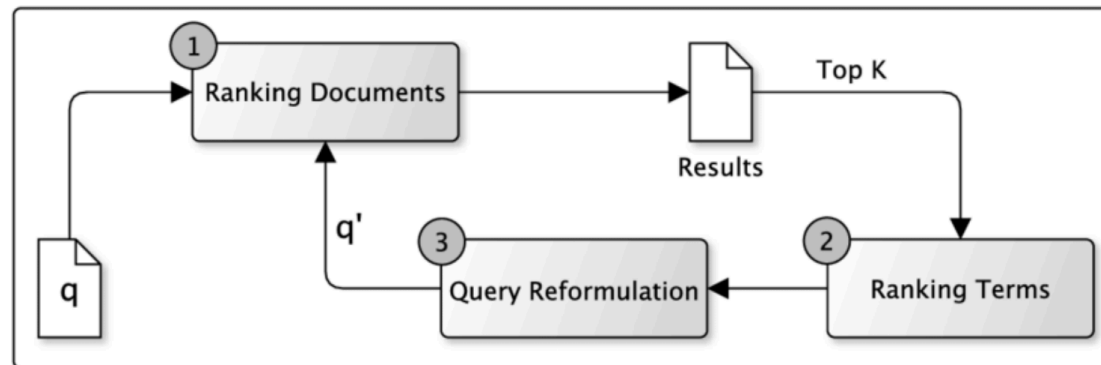


Applications

Information retrieval, textual inference

Related issues

efficiency (indexing, pre-selection)



🎯 Idea

Ranking documents = classifying a pair of texts

Applications

- Segmentation
- Named entity recognition
- Semantic annotation
- Question-Answering

When **Sebastian Thrun** PERSON started at **Google** ORG in **2007** DATE, few people outside of the valley took him seriously. "I can tell you very senior CEOs of major **American** NORP car companies would have taken his hand and turn away because I wasn't worth talking to," said **Thrun** PERSON, now the co-founder and CEO of online higher education startup Udacity, in an interview with **Recode** ORG **earlier this week** DATE.

A little **less than a decade later** DATE, dozens of self-driving startups have cropped up while auto industry around the world clamor, wallet in hand, to secure their place in the fast-moving world of fully autonomous transportation.

Passage Sentence

In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under gravity.

Question

What causes precipitation to fall?

Answer Candidate

gravity

🎯 The goal is to classify each word of the text

word 1	word 2	word 3	word 4	...
cl. 1	cl. 2	cl. 3	cl. 4	...



Information Extraction

We classify each word in the class B (begin) | E (end) | I (intermediate) - Type

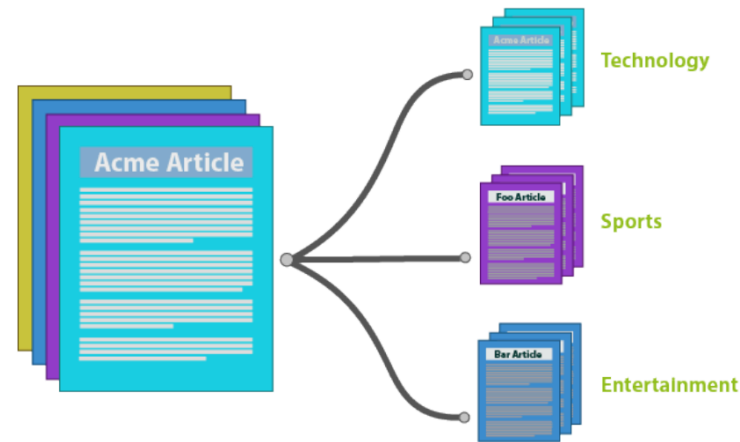
Sophie	went	to	Jean	Jaurès	Boulevard
EB-Name	NA	NA	B-Location	I-Location	E-Location

Result

- **Name:** Sophie
- **Location:** Jean-Jaurès Boulevard

Applications

- Text classification
- Sentiment detection



🎯 The goal is to classify a text

word 1	word 2	word 3	word 4	...
class				

🎯 Goal

The goal is to "decode a text"

Applications

- Machine translation
- Image captioning
- Dialogue
- Data-to-text
- ...

💡 Principle

$\mathbf{s}_t \in \mathbb{R}^n$ represents the text generated up to token t

- $\mathbf{s}_0$ represents the initial state (= question representation, etc.)
 - $\mathbf{s}_t$ is computed with a recurrent, convolutional, or self-attention network
- A state $\mathbf{s}_t$ defines a probability over words

$$\prod_t P(w_t | \mathbf{s}_t) \text{ with } \mathbf{s}_{t+1} = f(\mathbf{x}_t, \mathbf{s}_t)$$

Text characteristics: many different styles, symbol arbitrariness, compositionality, context

Tasks: Everything is *classification* (word-level, text-level, bi-text level)

! Consequences

Computing a good representation of text of words is key to any NLP task,

☞ but this is a **very difficult** thing...

☞ and this where models have struggled for a long time!

💡 Two main questions

Segmentation: What are the basic units of text?

Representation: How to represent them?

PRE-PROCESSING

► Text pre-processing

Whatever the NLP task... you need to cleaning and preparing raw text data for analysis or model training. There are usually three steps:

▶ Text pre-processing

Whatever the NLP task... you need to cleaning and preparing raw text data for analysis or model training. There are usually three steps:

✓ The different steps

Text pre-processing

Whatever the NLP task... you need to cleaning and preparing raw text data for analysis or model training. There are usually three steps:

✓ The different steps

1 Tokenization: breaking down into meaningful units

Whatever the NLP task ... you need to cleaning and
preparing raw text data for analysis or model training

Text pre-processing

Whatever the NLP task... you need to cleaning and preparing raw text data for analysis or model training. There are usually three steps:

✓ The different steps

1 Tokenization: breaking down into meaningful units

Whatever the NLP task ... you need to cleaning and
preparing raw text data for analysis or model training

2 Text removal: cleaning up "useless units"

Whatever ~~the~~ NLP task ~~...~~ ~~you~~ ~~need~~ ~~to~~ cleaning ~~and~~
preparing raw text data ~~for~~ analysis ~~or~~ model training

Text pre-processing

Whatever the NLP task... you need to cleaning and preparing raw text data for analysis or model training. There are usually three steps:

✓ The different steps

- 1 **Tokenization:** breaking down into meaningful units

Whatever the NLP task ... you need to cleaning and
preparing raw text data for analysis or model training

- 2 **Text removal:** cleaning up "useless units"

Whatever ~~the~~ NLP task ~~...~~ ~~you~~ ~~need~~ ~~to~~ cleaning ~~and~~
preparing raw text data ~~for~~ analysis ~~or~~ model training

- 3 **Normalization:** stemming, *lemmatization*, lowercasing...

whatever nlp task clean prepare raw text data
analysis model train

Text pre-processing

Whatever the NLP task... you need to cleaning and preparing raw text data for analysis or model training. There are usually three steps:

✓ The different steps

1 **Tokenization:** breaking down into meaningful units

Whatever the NLP task ... you need to cleaning and
preparing raw text data for analysis or model training

2 **Text removal:** cleaning up "useless units"

Whatever ~~the~~ NLP task ~~...~~ ~~you~~ ~~need~~ ~~to~~ cleaning ~~and~~
preparing raw text data ~~for~~ analysis ~~or~~ model training

3 **Normalization:** stemming, *lemmatization*, lowercasing...

whatever nlp task clean prepare raw text data
analysis model train

DISCUSSION: WHAT ARE THE LIMITS?

Word

- ☒ Most natural?
- ☐ Large vocabulary
- ☐ Unknown words

Word

- ☒ Most natural?
- ☐ Large vocabulary
- ☐ Unknown words

Character

- ☒ No unknown words
- ☒ Small vocabulary
- ☐ Less semantic / natural

Word

- ⊕ Most natural?
- ⊖ Large vocabulary
- ⊖ Unknown words

Character

- ⊕ No unknown words
- ⊕ Small vocabulary
- ⊖ Less semantic / natural

Sub-units (unsupervised algorithm)

Intermediate between word and character / trained unsupervised

Word

-  Most natural?
-  Large vocabulary
-  Unknown words

Character

-  No unknown words
-  Small vocabulary
-  Less semantic / natural

Sub-units (unsupervised algorithm)

Intermediate between word and character / trained unsupervised

Example

An osteopath can fix your back

 An / _osteo / path / _can / _fix / _your / _back /

PRE-PROCESSING ► TOKENIZATION ALGORITHMS: BYTE PAIR ENCODING (BPE)

 Sennrich, R. et al. 2016. *Neural Machine Translation of Rare Words with Subword Units*. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.



How It Works

1. Start with a **character-level vocabulary** (or bytes for byte-level BPE)
2. Find the **most frequent adjacent pair** of tokens in the corpus
3. Merge that pair into a single new token
4. Repeat N times (typically 32,000 merges)



Example

Start: "l o w e r" (characters)

- Iteration 1: Most frequent pair is (l, o) → merge to "lo w e r"
- Iteration 2: Most frequent pair is (lo, w) → merge to "low e r"
- Iteration 3: Most frequent pair is (low, e) → merge to "lowr"



Key Characteristic

Greedy, frequency-based: Always picks the globally most frequent pair

Used in: GPT-2, GPT-3, NeoBERT, ...

 Schuster, M. and Nakajima, K. 2012. Japanese and Korean Voice Search. 2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).



How It Works

1. Start with a **character-level vocabulary**
2. For each possible pair, compute a **likelihood score**
3. Merge the pair with the **highest likelihood** (not frequency)
4. Repeat N times



Likelihood Formula

Uses $\mathbb{W}$ **Pointwise mutual information** between the two tokens

$$\text{PMI} = \log \frac{P(AB)}{P(A\bullet) \times P(\bullet B)} = \log \frac{\text{count}(\text{pair})}{\text{count}(\text{first}) \times \text{count}(\text{second})}$$



Key Characteristic

Probabilistic, likelihood-based: Finds more linguistically meaningful splits

Used in: BERT, DistilBERT, RoBERTa, ...

 Kudo, T. 2018. *Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates*.

Unigram Model

Assumes that a segmentation is modeled by a unigram distribution

$$P(\mathbf{x}) = \prod_{i=1}^M p(x_i) \text{ for a segmentation } \mathbf{x} = (x_1, \dots, x_M)$$

The likelihood of a dataset D is given by

$$\mathcal{L} = \sum_{s=1}^{|D|} \log \left(P \left(X^{(s)} \right) \right) = \sum_{s=1}^{|D|} \log \left(\sum_{\mathbf{x} \in \mathcal{S}(X^{(s)})} P(\mathbf{x}) \right)$$

Removing tokens

1. Start with a **large initial vocabulary** $\{x_i\}$
2. Iteratively **remove the token** that increases overall loss the least
3. Continue until target vocabulary size is reached

Example

Text: "reading"

SentencePiece (BPE mode):

- Raw input: "_reading" (space is explicit character)
- Tokens: ['_read', 'ing']

Key Characteristic

Language-independent, no pre-processing: Works on any language without language-specific preprocessing

Used in: T5, mT5, XLNet, LLaMA, multilingual models

Aspect	BPE	WordPiece	SentencePiece
Selection Criterion	Most frequent pair	Highest likelihood	Remove lowest-loss token
Pre-tokenization	Needed (whitespace)	Needed (whitespace)	Not needed
Space Handling	Implicit or Ġ marker	Implicit or ## marker	Explicit _ marker
Language Support	English-focused	English-focused	Multilingual
Merging Direction	Bottom-up (char → word)	Bottom-up (char → word)	Top-down (many → few)

Examples of BPE vs SentencePiece (Unigram) segmentation

Original: furiously	Original: tricycles	Original: nanotechnology
BPE: _fur iously	BPE: _t ric y cles	BPE: _n an ote chn ology
Uni. LM: _fur ious ly	Uni. LM: _tri cycle s	Uni. LM: _nano technology

Original: Completely preposterous suggestions
BPE: _Comple t ely _prep ost erous _suggest ions
Unigram LM: _Complete ly _pre post er ous _suggestion s

Original: corrupted	Original: 1848 and 1852,
BPE: _cor rupted	BPE: _184 8 _and _185 2,
Unigram LM: _corrupt ed	Unigram LM: _1848 _and _1852 ,

Original	磁性は様々な分類がなされている。				
BPE	磁	性	は	様々	に分類がなされている。
Unigram LM	磁	性	は	様々	に 分類がなされている。
Gloss	magnetism	(top.)	various ways	in	classification is done .
Translation	Magnetism is classified in various ways.				

📖 Bostrom, K. and Durrett, G. 2020. Byte Pair Encoding Is Suboptimal for Language Model Pretraining. Findings of the Association for Computational Linguistics: EMNLP 2020.

! No guarantee that the sub-tokens are "meaningful"

🔗 [Dynamic Chunking \(H-Net\) : A New Approach to Tokenizer-Free AI Text Processing](#)

! Junk Tokens

After merging (BPE/WordPiece), some tokens might be left-over. Why?

📖 Bostrom, K. and Durrett, G. 2020. *Byte Pair Encoding Is Suboptimal for Language Model Pretraining. Findings of the Association for Computational Linguistics: EMNLP 2020.*

! Multiple segmentation

Multiple segmentation can lead to the same text

📖 Kudo, T. 2018. *Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates.*

! Multilinguality

Vocabulary can get big when tokenizing texts in multiple languages (and tokens even less meaningful)

! No guarantee that the sub-tokens are "meaningful"

🔗 [Dynamic Chunking \(H-Net\) : A New Approach to Tokenizer-Free AI Text Processing](#)

! Junk Tokens

After merging (BPE/WordPiece), some tokens might be left-over. Why?

📖 *Bostrom, K. and Durrett, G. 2020. Byte Pair Encoding Is Suboptimal for Language Model Pretraining. Findings of the Association for Computational Linguistics: EMNLP 2020.*

🔑 Some works still study how to deal with character/byte-level tokenization, more on this week 3!

Non exhaustive list:

📖 *Dynamic Chunking for End-to-End Hierarchical Sequence Modeling: 2025. <http://arxiv.org/abs/2507.07955>.*

📖 *Byte Latent Transformer: Patches Scale Better Than Tokens: 2024. <http://arxiv.org/abs/2412.09871>.*

! Multilinguality

Vocabulary can get big when tokenizing texts in multiple languages (and tokens even less meaningful)

TRAINING

- 1 **Training:** Training a tokenizer on a representative set of texts (**WordPiece**, **BPE** or **SentencePiece**)

INFERENCE

- 1 **Tokenization:** breaking down text into meaningful units
- 2 ~~**Text removal:** cleaning up "useless units"~~
- 3 **Normalization:** lowercasing (uncased) or nothing (cased)

DISTRIBUTED WORD REPRESENTATIONS

How to process documents?

Bag-of-words approach

ce nouvel iPhone, quelle merveille



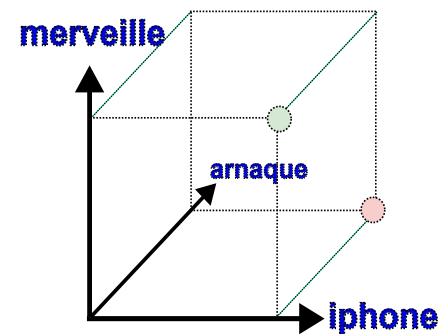
un iPhone? quelle arnaque!



merveille nouvel ce iPhone quelle un arnaque

1 1 1 1 1 0 0

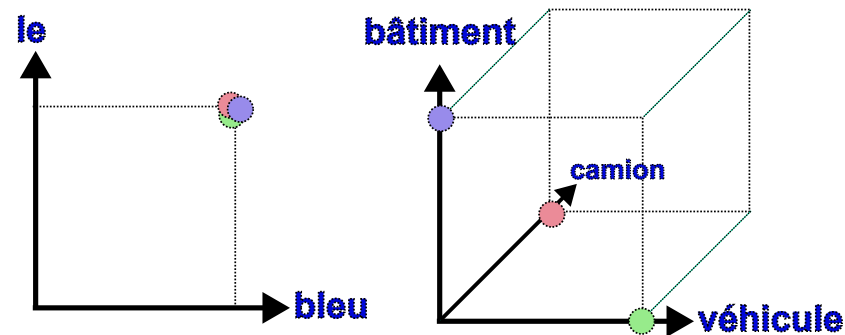
0 0 0 1 1 1 1



SIMILARITY = DOT PRODUCT

sum of products of weights in each dimension

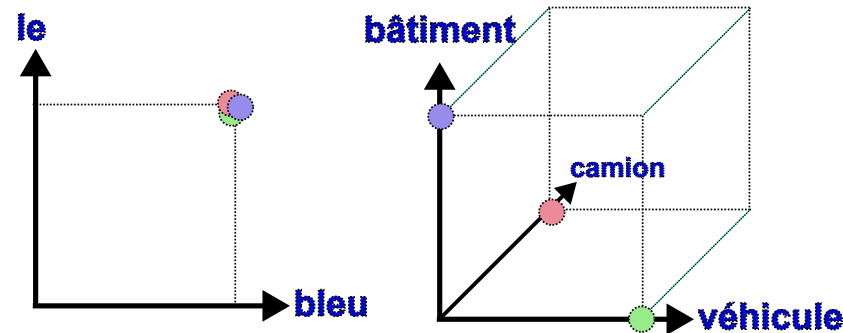
	<i>the</i>	<i>vehicle</i>	<i>building</i>	<i>blue</i>	<i>truck</i>
● the blue vehicle	1	1	0	1	0
● the blue building	1	0	0	1	0
● the blue truck	1	0	0	1	1



SIMILARITY = DOT PRODUCT

sum of products of weights in each dimension

	<i>the</i>	<i>vehicle</i>	<i>building</i>	<i>blue</i>	<i>truck</i>
● the blue vehicle	1	1	0	1	0
● the blue building	1	0	0	1	0
● the blue truck	1	0	0	1	1



⊕ "the blue truck" . "the blue vehicle" = 2

⊖ "the blue truck" . "the blue building" = 2

MOVING TO A SEMANTIC SPACE!

...

Vehicle	Transport	Color	Color	Construction	...	...
truck	0.5	0.5	0	0	0	...
vehicle	1	0	0	0.1	0	...
blue	0	0	0.5	0.7	0	...
building	0	0	0	0.1	0.8	...

MOVING TO A SEMANTIC SPACE!

...

Vehicle	Transport	Color	Color	Construction	...	...
truck	0.5	0.5	0	0	0	...
vehicle	1	0	0	0.1	0	...
blue	0	0	0.5	0.7	0	...
building	0	0	0	0.1	0.8	...

We can now compare words (and sentences)



A solution

Exploit regularities

Representation in a vector space $\mathbb{R}^n$



Precursors : Saussure/Wittgenstein, then Zellig Harris and John Firth



Word sequence

The boy went...

- to the dentist
- to school
- to the moon
- ...

In practice, simpler = word co-occurrences

Loading...

Representation

$$\text{cat} = (\underbrace{-0.15, -0.02, -0.23, -0.23, -0.18, \dots}_{n \text{ numbers}})$$

$$\text{dog} = (\underbrace{-0.40, 0.37, 0.02, -0.34, 0.05, \dots}_{n \text{ numbers}})$$

$$\text{computer} = (\underbrace{-0.27, -0.14, 0.23, -0.26, 0.09, \dots}_{n \text{ numbers}})$$

Similarity

$$s(\text{cat}, \text{dog}) = (-0.15 \times -0.40) + (-0.02 \times 0.37) + (-0.23 \times -0.02) + \dots = 0.8$$

$$s(\text{cat}, \text{computer}) = (-0.15 \times -0.27) + (-0.02 \times -0.14) + (-0.23 \times 0.23) + \dots = 0.2$$

DISTRIBUTED WORD REPRESENTATIONS ► WORD LEARNING

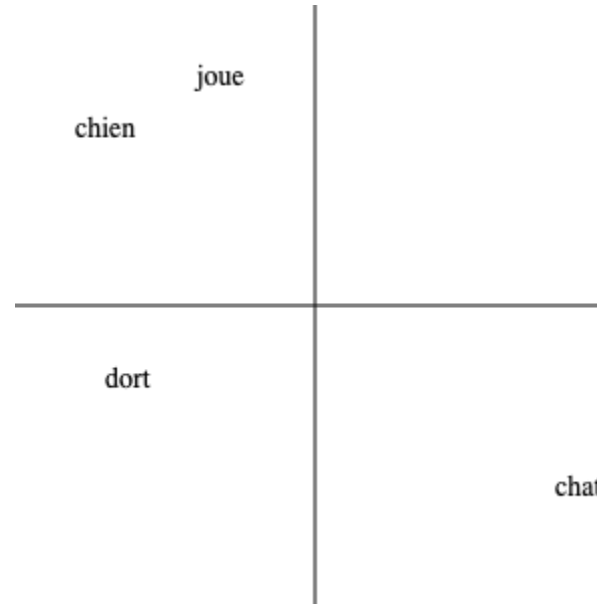
REPRÉSENTATIONS

chat	1,74	-1,10
dort	-1,24	-0,35
chien	-1,44	1,30
joue	-0,63	1,66

PROBABILITÉS

chat dort chien joue

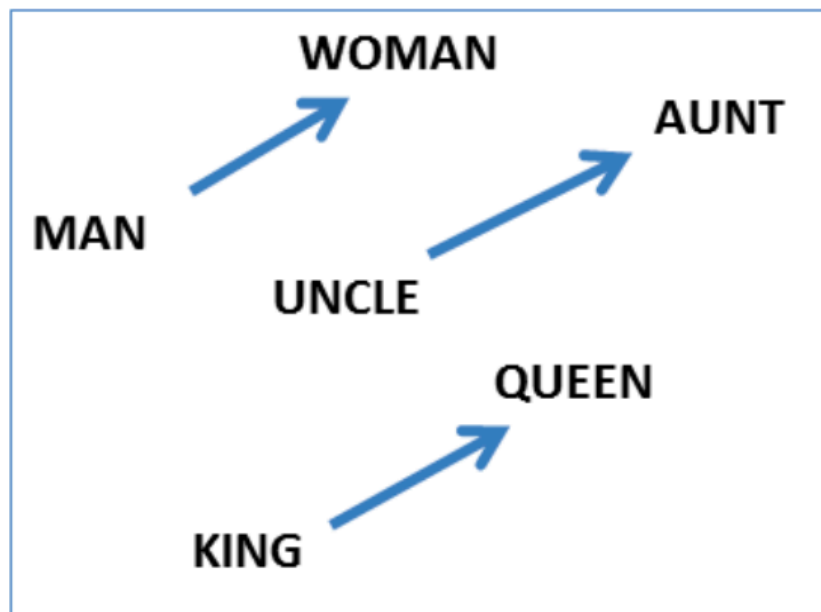
chat		0,98	0,00	0,02
dort	0,00		0,83	0,16
chien	0,00	0,12		0,88
joue	0,00	0,03	0,97	



Vitesse 0.01

Cible : Contexte :

Fonction objectif ↓ l : ?



💡 Transformation = Relation

A **geometric** transformation corresponds to a **semantic** relation

Example

$$x_{\text{man}} = x_{\text{woman}} + t_{\text{womanhood}}$$

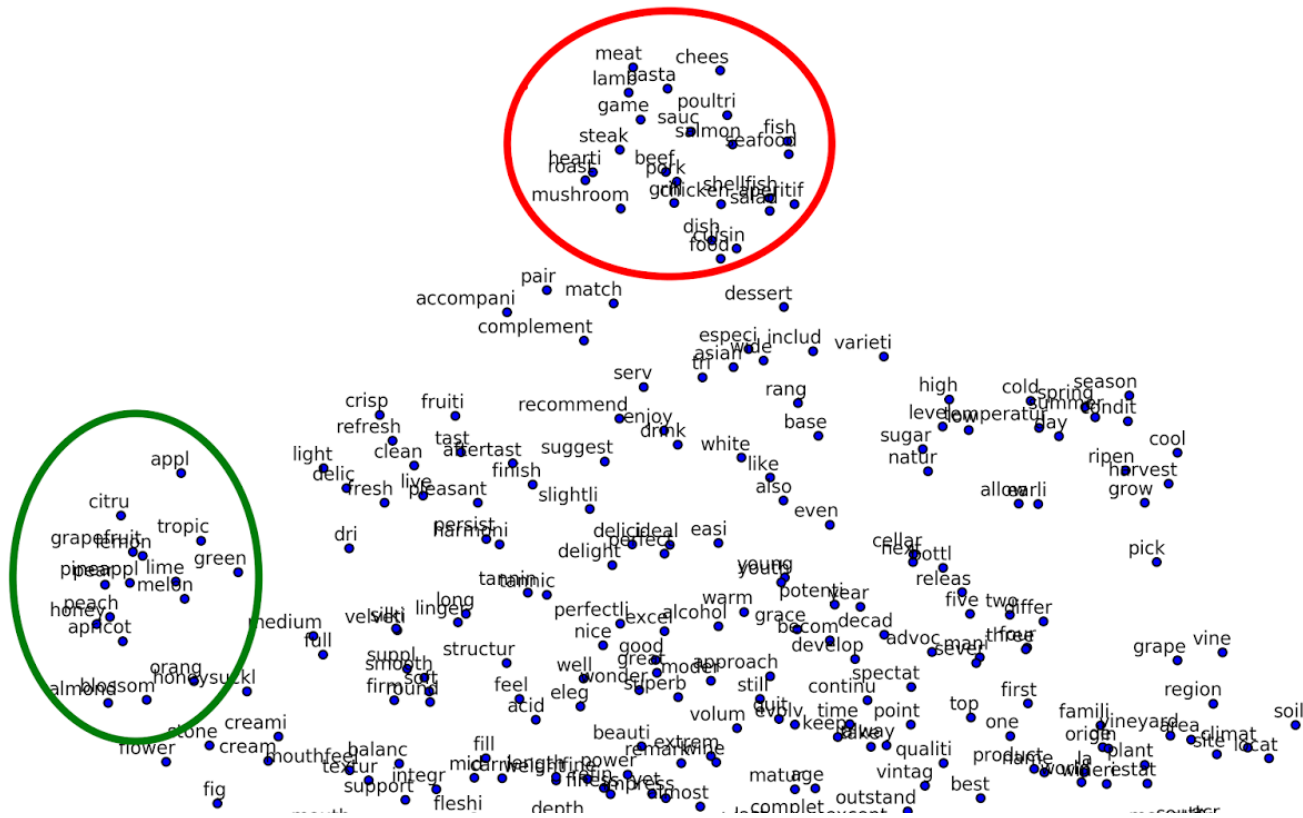
$$x_{\text{uncle}} = x_{\text{aunt}} + t_{\text{womanhood}}$$

...

These regularities are identified a posteriori from the learning.

📖 Mikolov, T. et al. 2013. *Distributed Representations of Words and Phrases and Their Compositionality*. NIPS'14.

DISTRIBUTED WORD REPRESENTATIONS ► REPRESENTATION SPACES: SIMILARITIES



A NICE BRAND NEW REPRESENTATION SPACE

For a word $x \in \mathcal{V}$, we associate a vector $\hat{m}$

$$x : m \in \mathcal{V} \rightarrow \hat{m}_1 \in \mathbb{R}^d$$

A NICE BRAND NEW REPRESENTATION SPACE

For a word $x \in \mathcal{V}$, we associate a vector $\hat{m}$

$$x : m \in \mathcal{V} \rightarrow \hat{m}_1 \in \mathbb{R}^d$$

We can now compare two words

We define similarity by the dot product

$$s(m_1, m_2) = x(m_1) \cdot x(m_2)$$

A NICE BRAND NEW REPRESENTATION SPACE

For a word $x \in \mathcal{V}$, we associate a vector $\hat{m}$

$$x : m \in \mathcal{V} \rightarrow \hat{m}_1 \in \mathbb{R}^d$$

We can now compare two words

We define similarity by the dot product

$$s(m_1, m_2) = x(m_1) \cdot x(m_2)$$

... and generate them

But we can also use this metric to define which word to generate

Idea = we want to generate a word corresponding to $x \in \mathbb{R}^d$

1. We compute $x \cdot \hat{m}$ for every word in the vocabulary
2. Then we normalize ($-x \cdot \hat{m}$ is the term's energy)

$$P(\text{generate } m|x) = \frac{\exp(x \cdot \hat{m})}{\sum_{m'} \exp(x \cdot \hat{m}')}$$

THERE ARE OTHER WORD REPRESENTATION MODELS

NON-CONTEXTUAL

- Glove

Global vectors learned from word co-occurrence statistics

 Pennington, J. et al. 2014. *Glove: Global Vectors for Word Representation*.

- FastText

Word embeddings enhanced with subword information (character n-grams)

 Joulin, A. et al. 2016. *Bag of Tricks for Efficient Text Classification*.

CONTEXTUAL

- Elmo

Embeddings from Language Models using bidirectional LSTMs

 Peters, M.E. et al. 2018. *Deep Contextualized Word Representations*. *NAACL*.

- Transformer architectures, e.g. BERT

Deep bidirectional representations using masked language modeling

 Devlin, J. et al. 2018. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.

NEURAL MODELS FOR NATURAL LANGUAGE PROCESSING

Currently, there are 4 solutions to represent sentences and/or solve text tasks:

Recurrent Networks

Processing sequences unit by unit

Possibility of using memories (*attention*)

Use cases: any task

Convolutional Networks

Hierarchical pattern detection

Use cases: classification (topic, sentiment)

Interaction Models

Alignment of two texts

Use cases: Information retrieval, inference (is this the right answer?)

Self-Attention Models

Contextual representation

Use cases: any task

Encoder

Represents each token / text by a vector in $\mathbb{R}^d$

- ☞ (usually) bi-directional

- ☞ **tasks:** token or text-level classification

Decoder

Token/Text representation depends on the previous ones

- ☞ uni-directional

- ☞ **tasks:** token or text-level classification, generation

Encoder-Decoder

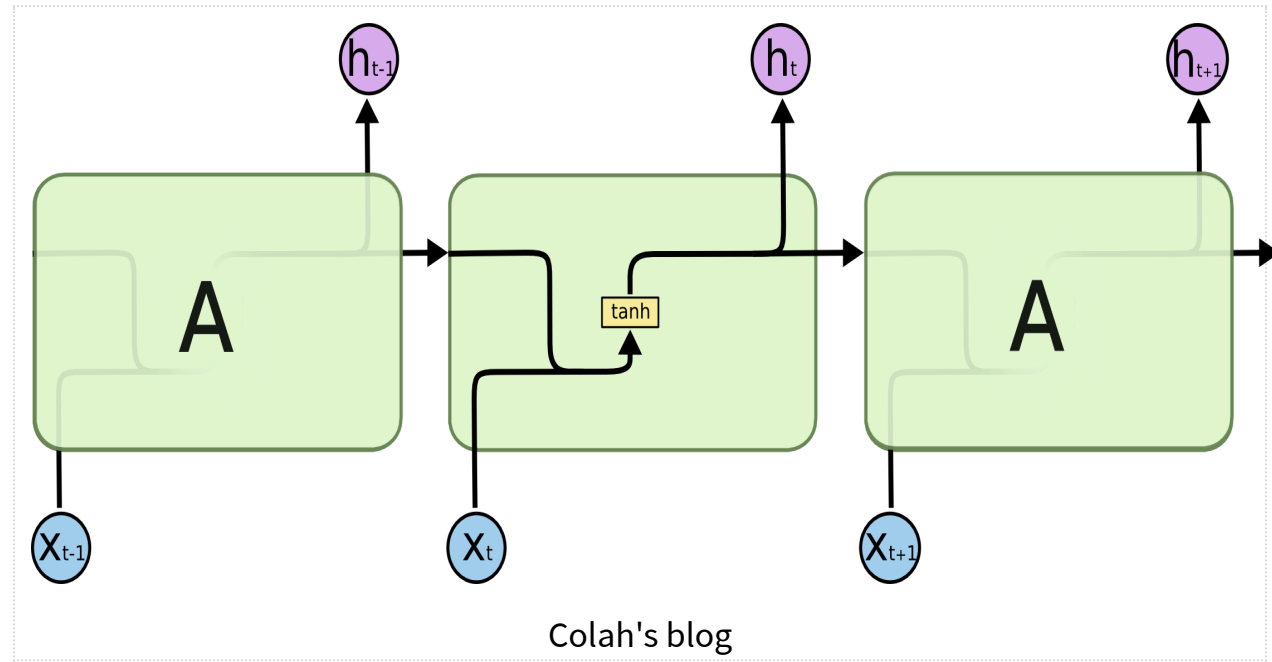
Combines an encoder (e.g. query, document to analyze) and a decoder (generation)

- ☞ **tasks:** Same as decoder

- ☞ Nowadays, mostly for multi-modal inputs

At each time t is associated

- A state $s_t \in \mathbb{R}^n$
- An input $x_t \in \mathbb{R}^p$



💡 Principle

The next state is computed from the input x_t and the previous state s_{t-1} :

$$s_t = f_{\theta}(s_{t-1}, x_t)$$

You've seen them in images... quick reminders

💡 Invariance hypothesis (by translation)

Processing **does not change** depending on position

- 🔧 In video, independence *by translation* (3D)
- 🔧 In images, independence *by translation* (2D)
- 🔧 In text, independence *by translation* (1D)

🧪 In text

Analyzing *I loved this movie* does not depend on context

Abstraction by composition

By composing convolutional layers, we detect increasingly complex patterns

 In images: edges, ..., objects

 In text, word groups, .., sentence fragments

In text

Analyzing *I loved this movie* can be done in two steps:

1. Detect

- "I loved" [ME/LIKE]
- "this movie" [OBJECT]

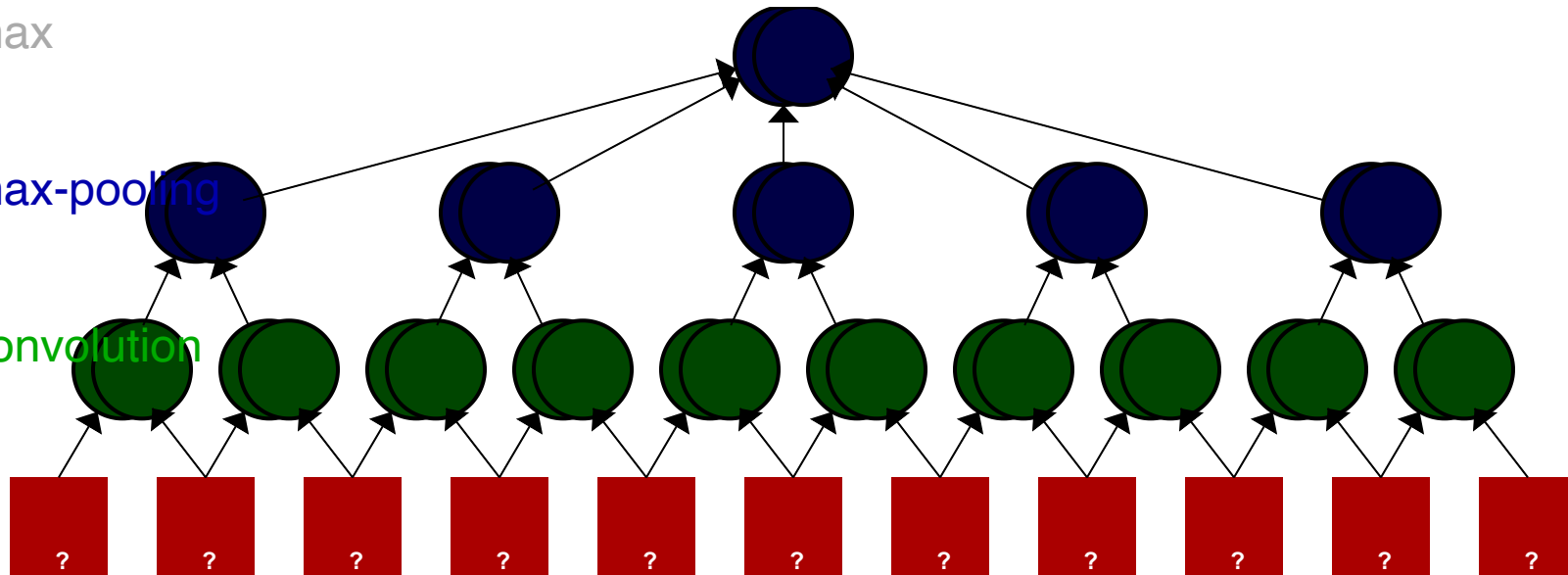
2. Detect [ME/LIKE] [OBJECT] = positive sentiment

Detect a pattern "I like"

max

max-pooling

convolution



We try to detect combinations of patterns in the sequence "don't like" AND "horror films" in the sequence

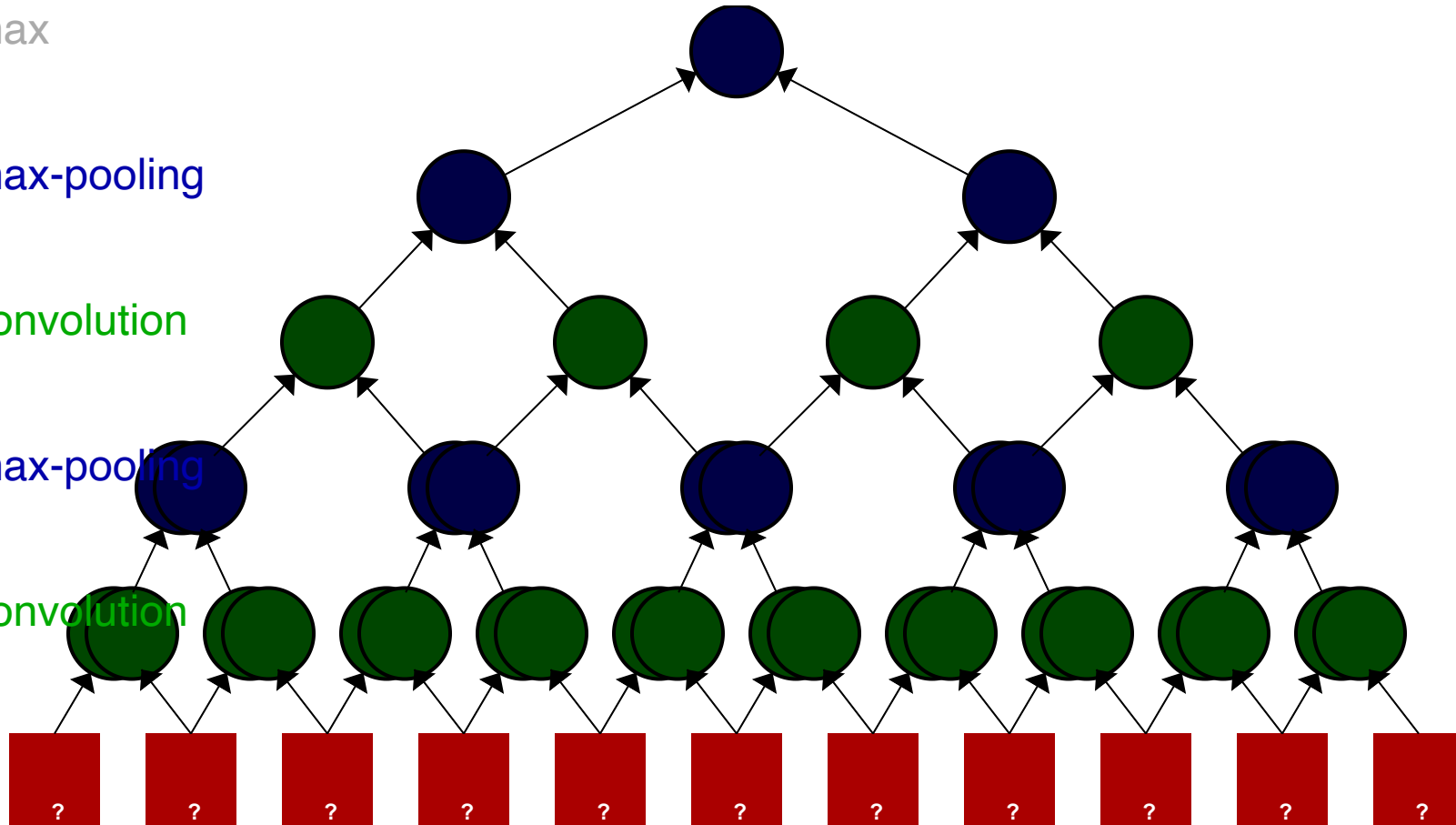
max

max-pooling

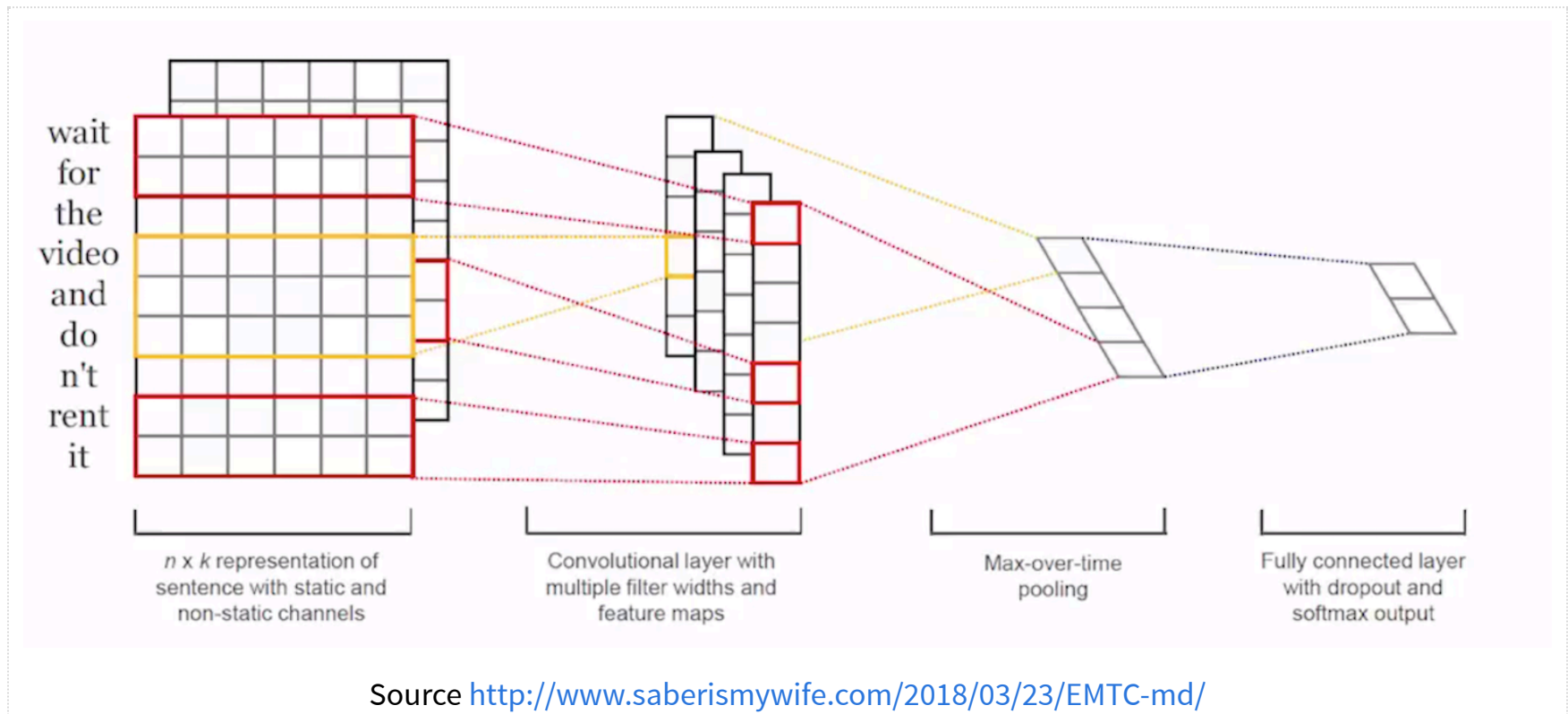
convolution

max-pooling

convolution



NEURAL MODELS FOR NATURAL LANGUAGE PROCESSING ► APPLICATION EXAMPLE: CLASSIFICATION



Recurrent Neural Networks (RNNs)

- **Sequential processing:** Cannot parallelize across time steps
Must process tokens one by one, slow for long sequences
- **Long-range dependencies:** Hard to capture distant relationships
Vanishing gradients despite LSTM/GRU improvements
- **Information bottleneck:** Fixed-size hidden state
All past information compressed into single vector

Recurrent Neural Networks (RNNs)

- **Sequential processing:** Cannot parallelize across time steps
Must process tokens one by one, slow for long sequences
- **Long-range dependencies:** Hard to capture distant relationships
Vanishing gradients despite LSTM/GRU improvements
- **Information bottleneck:** Fixed-size hidden state
All past information compressed into single vector

Convolutional Neural Networks (CNNs)

- **Limited receptive field:** Need many layers for long-range dependencies
Each layer only sees local context (e.g., 3-5 tokens)
- **Not position-aware:** Same pattern everywhere
Difficult to model positional relationships
- **Fixed pattern size:** Kernel size determines what patterns can be detected
Hard to capture variable-length dependencies

Recurrent Neural Networks (RNNs)

- **Sequential processing:** Cannot parallelize across time steps
Must process tokens one by one, slow for long sequences
- **Long-range dependencies:** Hard to capture distant relationships
Vanishing gradients despite LSTM/GRU improvements
- **Information bottleneck:** Fixed-size hidden state
All past information compressed into single vector

Convolutional Neural Networks (CNNs)

- **Limited receptive field:** Need many layers for long-range dependencies
Each layer only sees local context (e.g., 3-5 tokens)
- **Not position-aware:** Same pattern everywhere
Difficult to model positional relationships
- **Fixed pattern size:** Kernel size determines what patterns can be detected
Hard to capture variable-length dependencies



Solution: Self-attention mechanisms

NEURAL MODELS FOR NATURAL LANGUAGE PROCESSING ATTENTION AND MEMORY

Given the shortcomings of RNNs...

- Information bottleneck (fixed-size hidden state)
- Difficulty with long-range dependencies
- Sequential processing limitations

Given the shortcomings of RNNs...

- Information bottleneck (fixed-size hidden state)
- Difficulty with long-range dependencies
- Sequential processing limitations

INSPIRATION FROM HUMAN COGNITION

Working Memory

Humans don't compress everything into a single representation
We have a **working memory** that we can query when needed

A new approach

What if neural networks could use **external memory**?
Allow models to **store and retrieve** information dynamically

Given the shortcomings of RNNs...

- Information bottleneck (fixed-size hidden state)
- Difficulty with long-range dependencies
- Sequential processing limitations

INSPIRATION FROM HUMAN COGNITION

Working Memory

Humans don't compress everything into a single representation
We have a **working memory** that we can query when needed

A new approach

What if neural networks could use **external memory**?
Allow models to **store and retrieve** information dynamically

Neural Turing Machines & Memory Networks

📖 Graves, A. et al. 2014. *Neural Turing Machines*.

📖 Weston, J. et al. 2014. *Memory Networks*.

- Explicit external memory component
- Read (and write) operations controlled by **attention**
- Can learn algorithms and store/retrieve facts

Neural Turing Machines & Memory Networks

- 📖 Graves, A. et al. 2014. *Neural Turing Machines*.
- 📖 Weston, J. et al. 2014. *Memory Networks*.

- Explicit external memory component
- Read (and write) operations controlled by **attention**
- Can learn algorithms and store/retrieve facts

🧪 Example: Question Answering with Memory Networks

- 📖 Weston, J. et al. 2014. *Memory Networks*.

Story: "John went to the kitchen. Mary got the milk. John moved to the garden."

Question: "Where is John?"

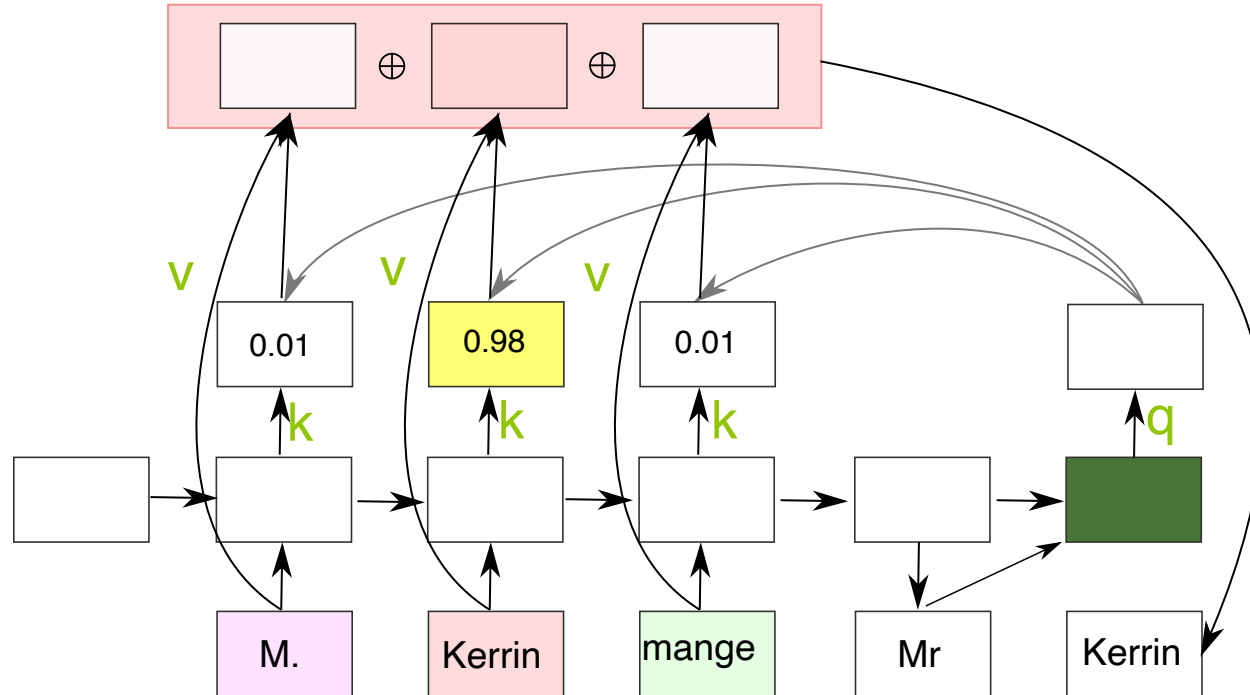
Memory: Store each sentence as a separate memory slot

Process: Query memory with the question → Retrieve relevant facts → Answer "garden"

🔑 The text itself can serve as memory

🧪 Translation

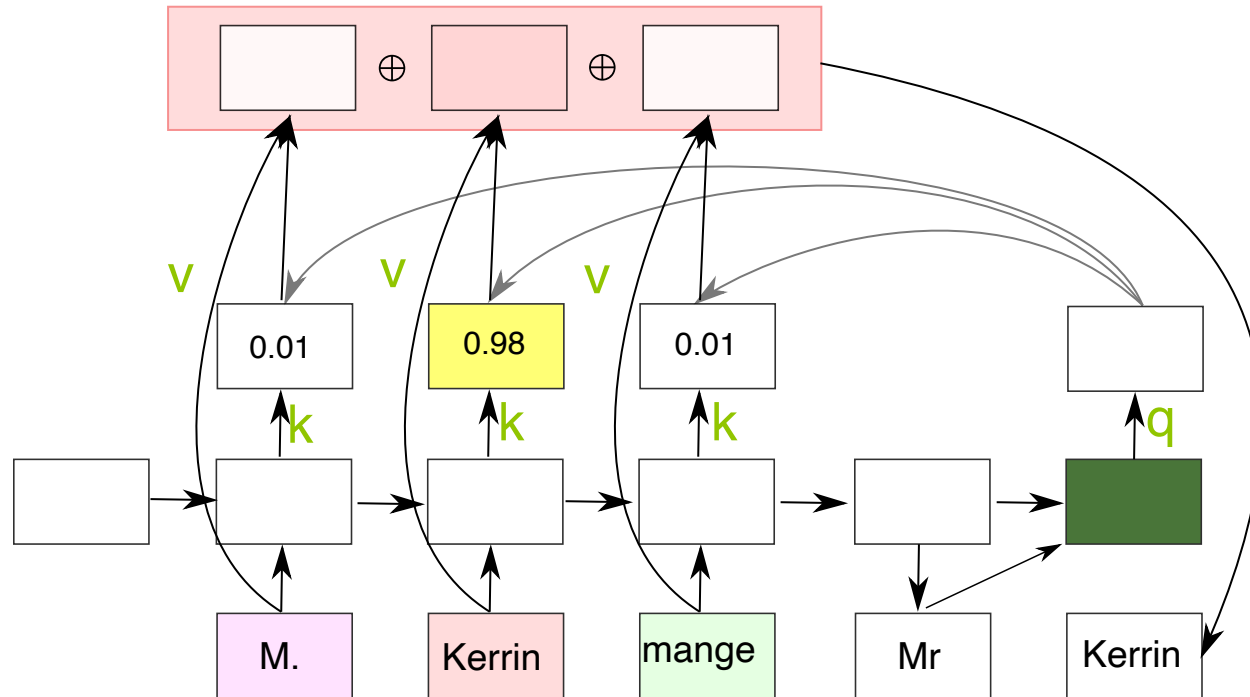
📖 Vinyals, O. et al. 2017. *Pointer Networks*.



🔑 The text itself can serve as memory

🧪 Translation

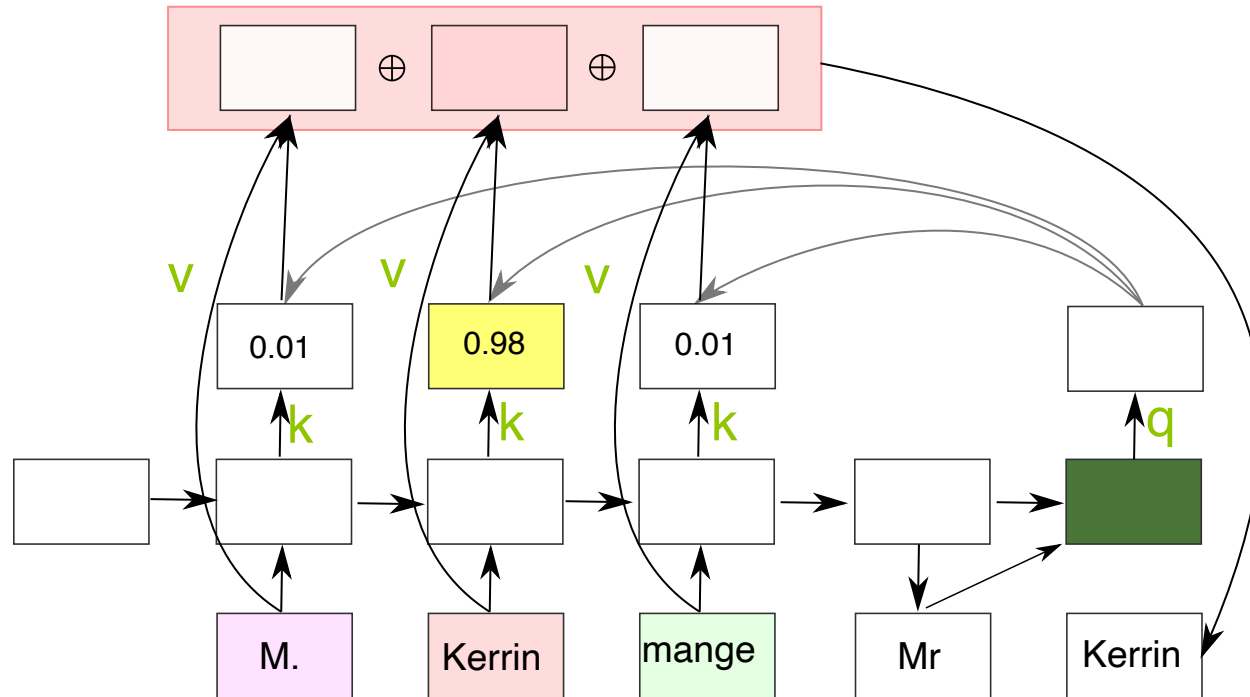
📖 Vinyals, O. et al. 2017. *Pointer Networks*.



🔑 The text itself can serve as memory

🧪 Translation

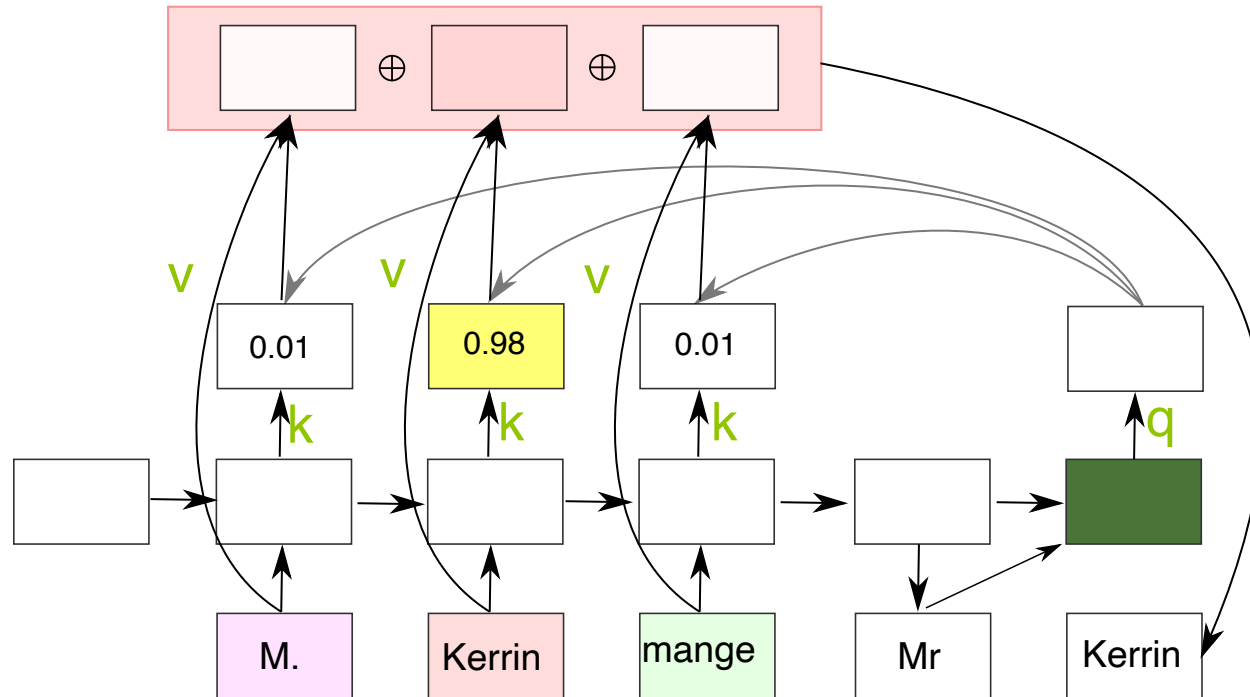
📖 Vinyals, O. et al. 2017. *Pointer Networks*.



🔑 The text itself can serve as memory

🧪 Translation

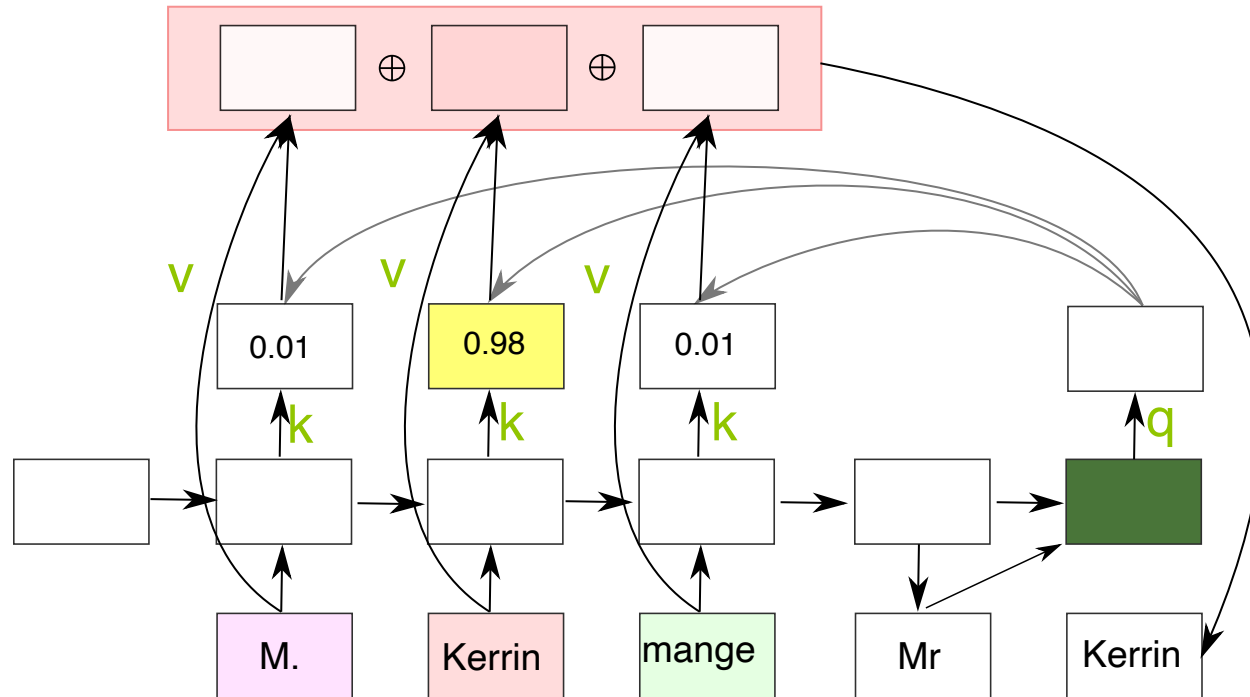
📖 Vinyals, O. et al. 2017. *Pointer Networks*.



🔑 The text itself can serve as memory

🧪 Translation

📖 Vinyals, O. et al. 2017. *Pointer Networks*.



Let a "memory" $M = \{\mathbf{m}_i \in \mathbb{R}^{n_m}\}$

We can extract from each memory element:

A key

a key $k(\mathbf{m}_i) \in \mathbb{R}^{n_k}$ that allows searching for a memory element

A value

a value $v(\mathbf{m}_i) \in \mathbb{R}^{n_v}$ that contains the value associated with this element

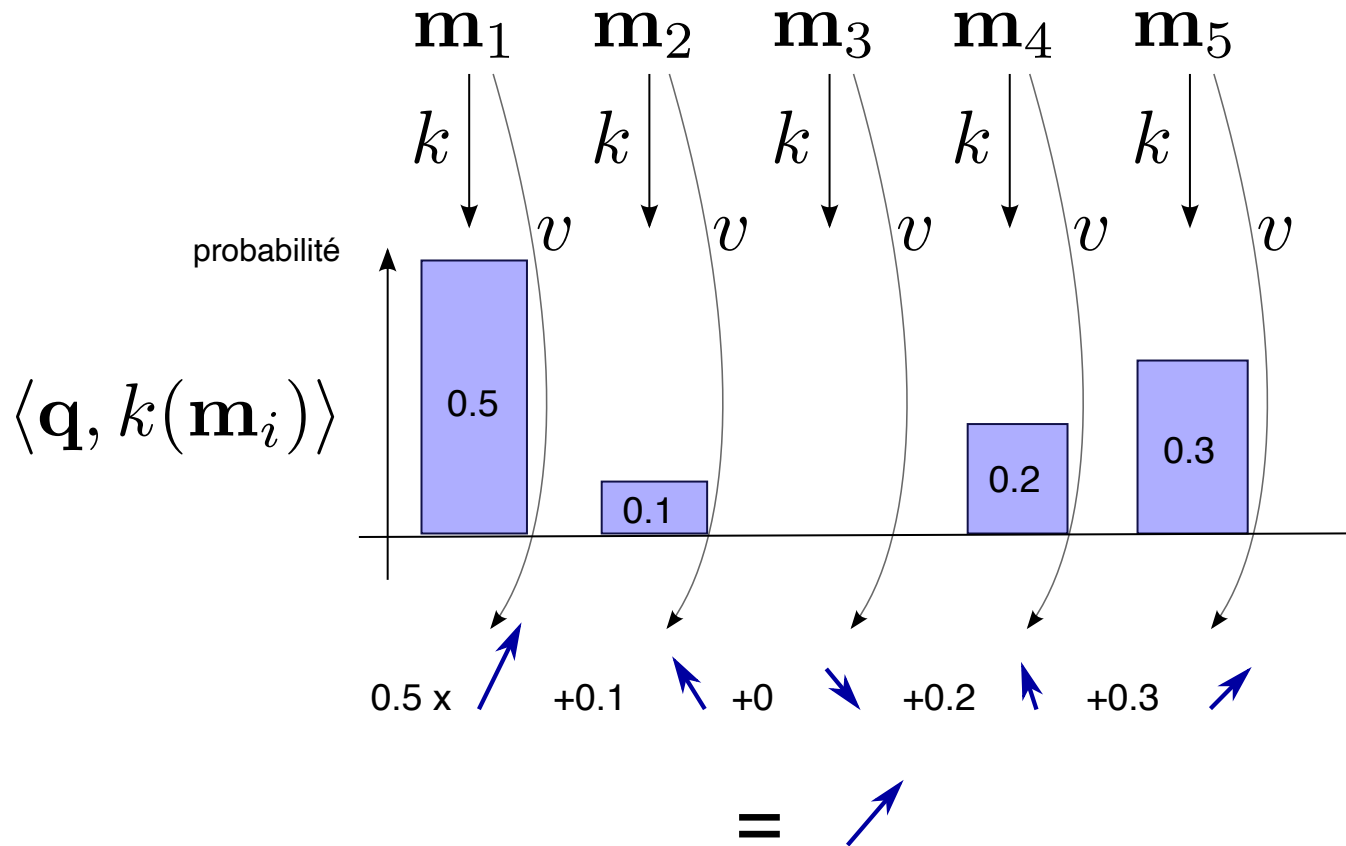
Given $\mathbf{x} \in \mathbb{R}^n$, we

1. Determine the query $q(\mathbf{x}) \in \mathbb{R}^{n_k}$
2. Compute an attention probability

$$\log p(A_i | \mathbf{x}, M) = K + k(\mathbf{m}_i) \cdot q(\mathbf{x})$$

3. Compute the output $f(\mathbf{x}) = \sum_i p(A_i) v(\mathbf{m}_i)$

Reading a memory composed of 5 units



Some attempts to build read/write memories for neural networks have been attempted using the same principles (with limited success)

📖 Graves, A. et al. 2014. *Neural Turing Machines*.

📖 Weston, J. et al. 2014. *Memory Networks*.

Some attempts to build read/write memories for neural networks have been attempted using the same principles (with limited success)

📖 Graves, A. et al. 2014. *Neural Turing Machines*.

📖 Weston, J. et al. 2014. *Memory Networks*.

memory-based networks have improved first automated translation (January 2017)

📖 Vinyals, O. et al. 2017. *Pointer Networks*.

Some attempts to build read/write memories for neural networks have been attempted using the same principles (with limited success)

📖 Graves, A. et al. 2014. *Neural Turing Machines*.

📖 Weston, J. et al. 2014. *Memory Networks*.

memory-based networks have improved first automated translation (January 2017)

📖 Vinyals, O. et al. 2017. *Pointer Networks*.

before the first paper on Transformer architectures appeared – also on translation (June 2017)

📖 Vaswani, A. et al. 2017. *Attention Is All You Need*.

TRANSFORMERS AND LLMS

📖 Vaswani, A. et al. 2017. *Attention Is All You Need*.

SOURCES

J. Alammam 2018 <http://jalammar.github.io/illustrated-transformer/>

P. Bloem 2019 <http://www.peterbloem.nl/blog/transformers>

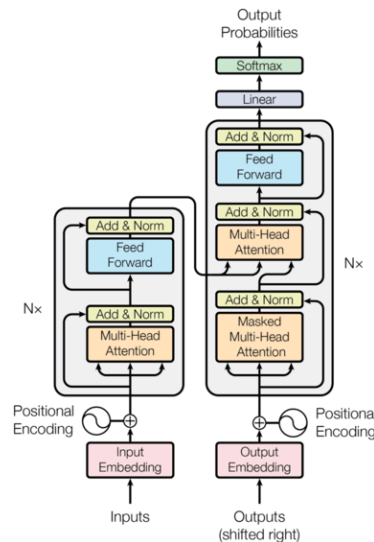


📖 Vaswani, A. et al. 2017. *Attention Is All You Need*.

SOURCES

J. Alammam 2018 <http://jalammar.github.io/illustrated-transformer/>

P. Bloem 2019 <http://www.peterbloem.nl/blog/transformers>



From the  Annotated
Tranformer

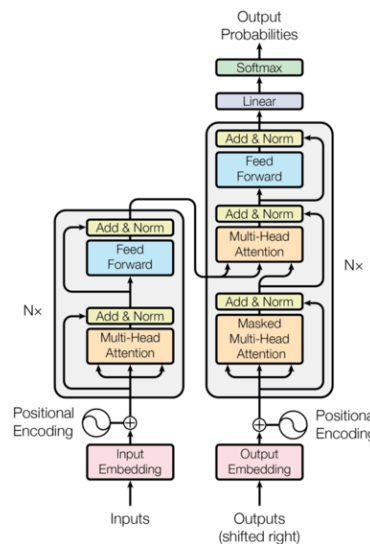
TRANSFORMERS AND LLMs ► SELF-ATTENTION MODEL

📖 Vaswani, A. et al. 2017. *Attention Is All You Need*.

SOURCES

J. Alammam 2018 <http://jalammar.github.io/illustrated-transformer/>

P. Bloem 2019 <http://www.peterbloem.nl/blog/transformers>



From the [Annotated Tranformer](#)

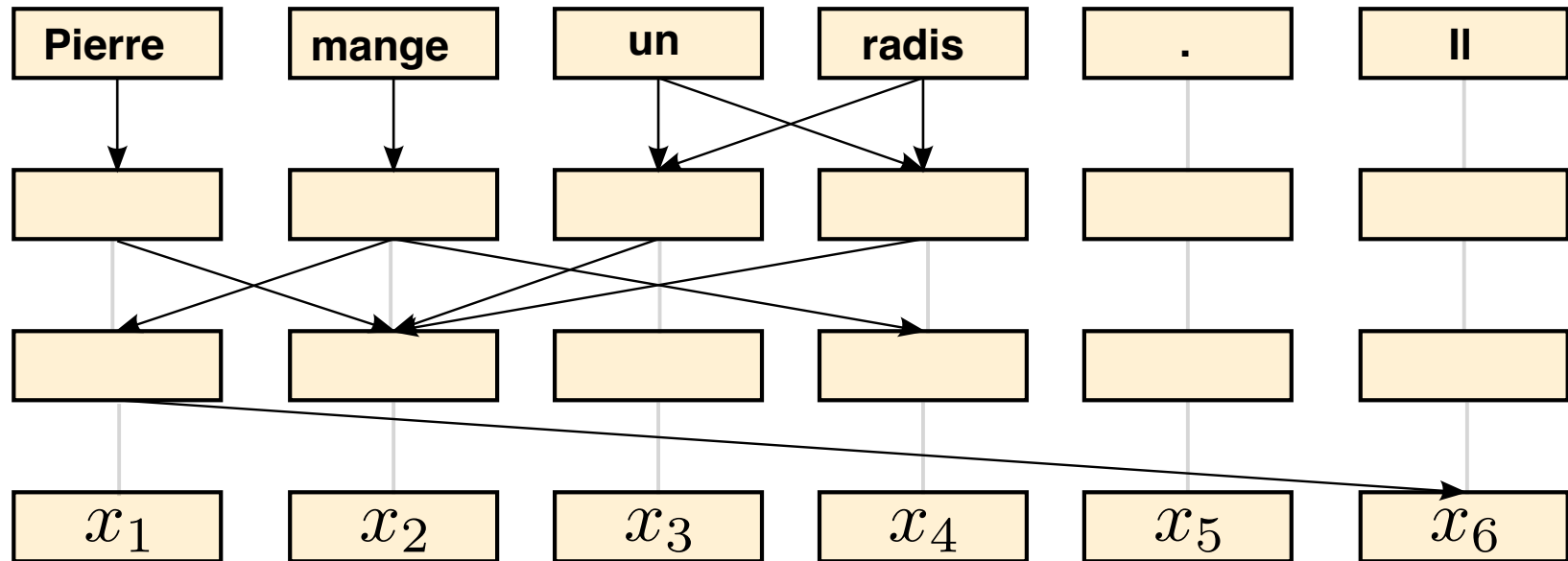
🔑 Big ideas

1. Text contextualization (memory, without recurrence)
The text *is* its memory
2. Limited vocabulary
With BPE initially
3. Pre-trained model (unsupervised) on large amounts of text
With BERT (encoder architecture)
4. Multi-task training
With

📖 Raffel, C. et al. 2019. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*.

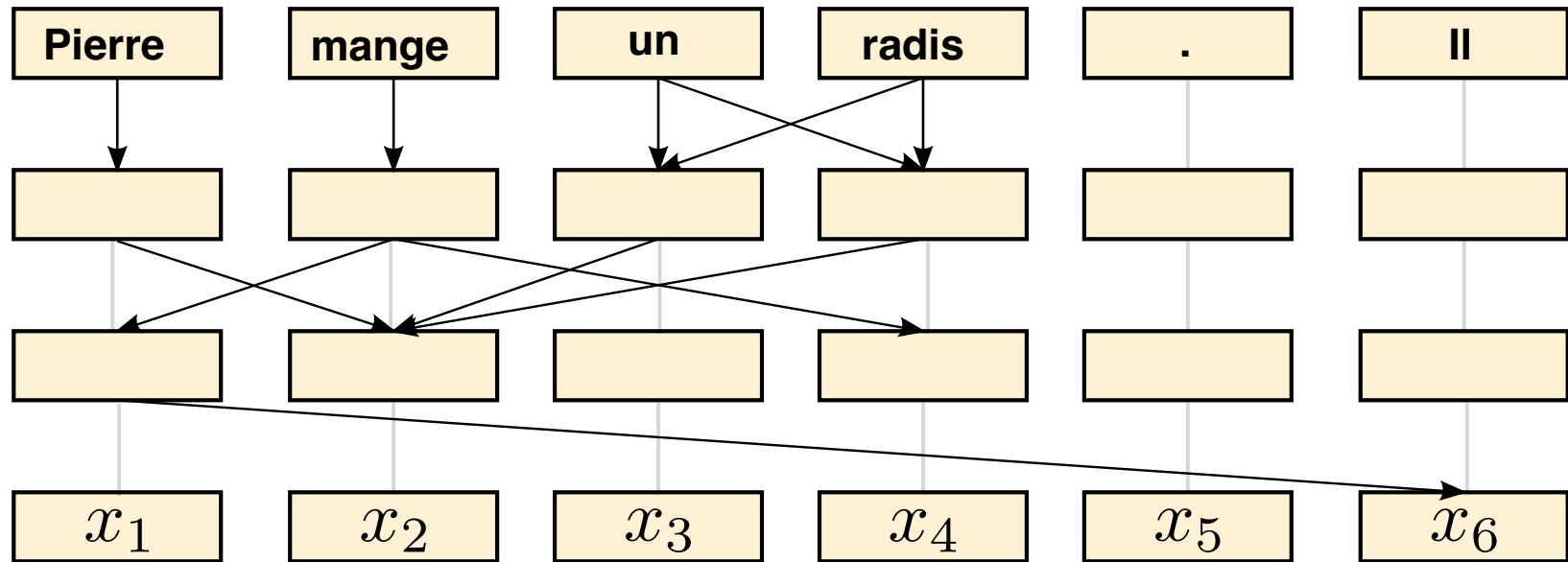
🔑 We use self-attention mechanisms (memory = text) that allow contextualizing *each* input token

📖 Vaswani, A. et al. 2017. *Attention Is All You Need*.



☞ We use self-attention mechanisms (memory = text) that allow contextualizing *each* input token

📖 Vaswani, A. et al. 2017. *Attention Is All You Need*.



☞ Input are tokens $t_1, \dots, t_6$

☞ We obtain *contextualized* representations $x_1, \dots, x_6$

🔗 Visualization with BertViz

► Pre-processing

Tokenization and normalization ➡ $\mathbf{t}_{1:L} = \mathbf{t}_1, \dots, \mathbf{t}_L \in \{1, \dots, V\}$

► Inputs

Initial input: from *tokens* $\mathbf{t}_{1:L}$ to *vectors* $\mathbf{x}_{1:L}^{(0)} = \mathbf{x}_1^{(0)}, \dots, \mathbf{x}_L^{(0)} \in \mathbb{R}^d$

► n layers

Each layer $\ell \in \{1, \dots, n\}$ is made of several block

- 1 Self-Attention $\mathbf{x}_{1:L}^{(\ell-1)} \rightarrow \mathbf{x}_{1:L}^{(\ell \angle SA)}$
- 2 Only for a decoder in an encoder-decoder
Attention on the encoder ➡ $\mathbf{x}_{1:L}^{(\ell \angle CA)}$
- 3 A two-layer feedforward neural network (*~ associative memory*)
 $\mathbf{x}_{1:L}^{(\ell-1 \angle A)} \rightarrow \mathbf{x}_{1:L}^{(\ell)}$

✓ The input

Uses an embedding table $\mathbf{r}_{sem}(\mathbf{t}_i) \in \mathbb{R}^d$

✓ The input

Uses an embedding table $\mathbf{r}_{sem}(\mathbf{t}_i) \in \mathbb{R}^d$

⚠ The self-attention process is invariant to permutations

How to include positional information?

✓ The input

Uses an embedding table $\mathbf{r}_{sem}(\mathbf{t}_i) \in \mathbb{R}^d$

⚠ The self-attention process is invariant to permutations

How to include positional information?

⚠ Absolute Positional Embeddings

Positional embedding $\mathbf{r}_{pos}(i) \in \mathbb{R}^d$

$\mathbf{r}_{pos}(i)$ can either be learnable parameters (BERT), or

$$PE_{(pos, 2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right)$$
$$PE_{(pos, 2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$

✓ The input

Uses an embedding table $\mathbf{r}_{sem}(\mathbf{t}_i) \in \mathbb{R}^d$

⚠ The self-attention process is invariant to permutations

How to include positional information?

⚠ Absolute Positional Embeddings

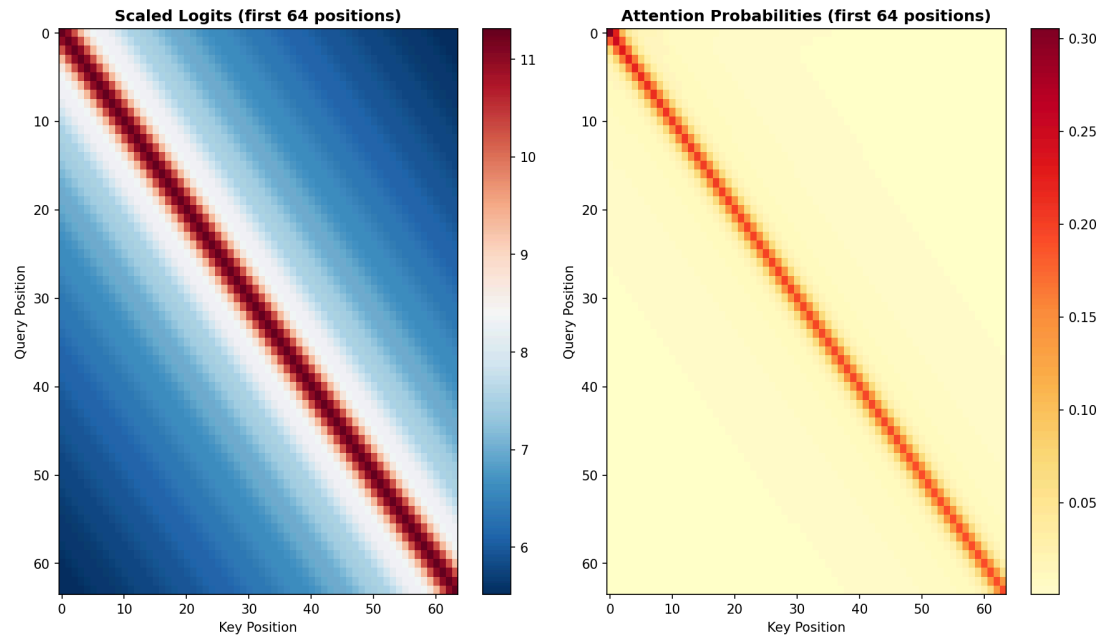
Positional embedding $\mathbf{r}_{pos}(i) \in \mathbb{R}^d$

$\mathbf{r}_{pos}(i)$ can either be learnable parameters (BERT), or

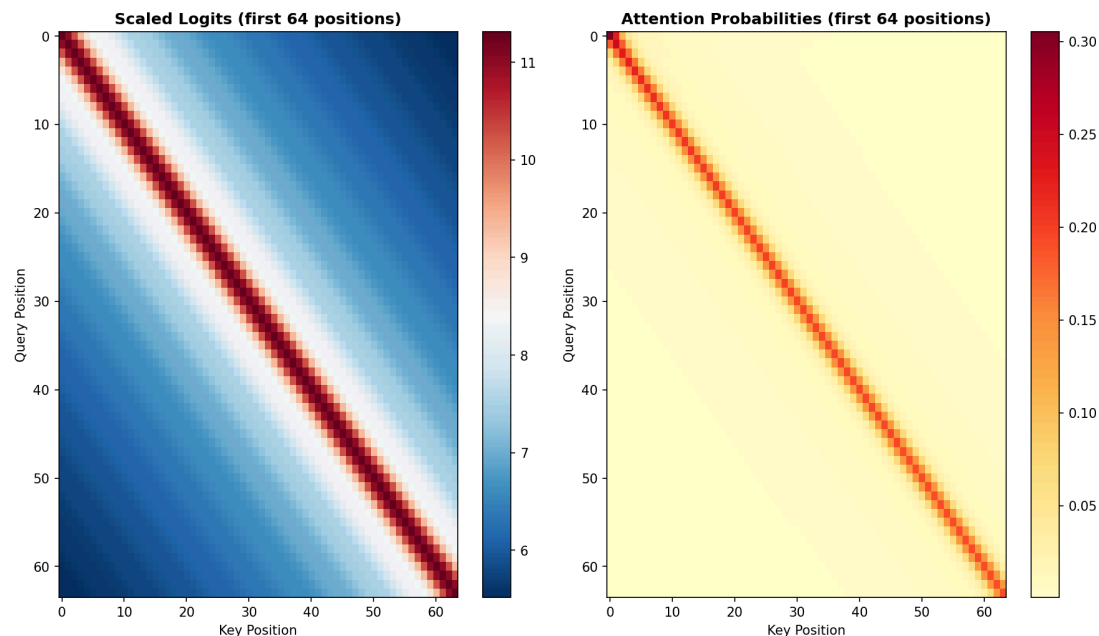
$$PE_{(pos, 2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right)$$
$$PE_{(pos, 2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$

The input of a transformer is then $\mathbf{r}_{pos}(i) + \mathbf{r}_{sem}(\mathbf{t}_i)$

🧪 Attention map for positional embeddings



Attention map for positional embeddings



Observations

- **Periodic pattern:** Diagonal stripes with regular spacing
- **Symmetry:** Pattern is symmetric around the diagonal
- **Decay:** Attention strength decreases with distance

Multi-Head Attention $\mathbf{z}_{1,...,L} = MHA(\mathbf{x}_{1...L}, \mathbf{y}_{1...L'})$

For each head h , we compute the attention probability:

$$\alpha_{ij}^h = \frac{1}{\sqrt{d_h}} \mathbf{Q}_h \mathbf{x}_i \cdot \mathbf{K}_h \mathbf{y}_j \quad \Rightarrow \quad p(j|i, h) = \text{softmax}_j(\alpha_{i\bullet})$$

Finally, we can compute the output of the the attention as:

$$\mathbf{z}_i = \sum_h \mathbf{O}_h \sum_j p(j|i, h) \mathbf{V}_h \mathbf{y}_j \text{ with } \mathbf{V}_h \mathbf{y}_j \in \mathbb{R}^{d_h}$$

Self-Attention and Cross-Attention Layer

$$\mathbf{x}_{1:L}^{(\ell_{\angle SA})} = LN \left(\mathbf{x}_{1:L}^{(\ell-1)} + MHA(\mathbf{x}_{1:L}^{(\ell-1)}, \mathbf{x}_{1:L}^{(\ell-1)}) \right)$$

with LN a layer normalization: $LN(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sigma} + \beta$

In the case of **cross-attention** in decoder (n' the number of layer of the encoder)

$$\mathbf{x}_{1:L}^{(\ell_{\angle CA})} = LN \left(\mathbf{x}_{1:L}^{(\ell-1)} + MHA(\mathbf{x}_{1:L}^{(\ell-1)}, \mathbf{y}_{1...L'}^{(n')}) \right)$$

► Causal mask

For autoregressive models (e.g., GPT), we mask future positions

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases}$$

The attention scores become: $\alpha_{ij}^h = \mathbf{Q}_h \mathbf{x}_i \cdot \mathbf{K}_h \mathbf{y}_j + M_{ij}$

► Example for sequence length 4

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

After softmax, positions only attend to current and previous positions

After attention aggregates information across tokens, we need to process each position independently



Role of the FFN

- Attention: **communication** between positions
- **FFN: computation** within each position
- Each token is processed identically but independently

After attention aggregates information across tokens, we need to process each position independently

💡 Role of the FFN

- Attention: **communication** between positions
- **FFN: computation** within each position
- Each token is processed identically but independently

📌 FFN: Token-wise operation

FFN is a two-layer neural network

$$\mathbf{z} = \mathbf{V}\phi(\mathbf{U}\mathbf{x} + \mathbf{b}) + \mathbf{c}$$

with $\mathbf{W}_1 \in \mathbb{R}^{d_1}$, $d_1 \gg d$ (e.g. $d_1 = 4d$ in BERT) and ϕ an activation function, e.g. in BERT a ReLU $\phi(\mathbf{x}) = \max(\mathbf{x}, 0)$

🔗 The i th output of the overall FFN network:

$$y_i = \sum_i \text{ReLU}(\mathbf{U}\mathbf{x} + \mathbf{b})_i \mathbf{V}_{\bullet i} + \mathbf{c}_i$$

SINCE 2018, A NUMBER OF CHANGES HAVE OCCURRED

1. Relative Positional Embeddings: ROPE, ALiBi
2. Pre-normalization to post-normalization
3. Shift from LayerNorm to RMSNorm
4. FFN activation function: from ReLU to SWIGLU
5. Sparse Attention Patterns

The problem with absolute positions

- Absolute positional embeddings (sinusoidal or learned) encode **where** a token is
- But often what matters is the **relative distance** between tokens
- Example: "the cat" should have similar representations whether at positions (0,1) or (10,11)

The problem with absolute positions

- Absolute positional embeddings (sinusoidal or learned) encode **where** a token is
- But often what matters is the **relative distance** between tokens
- Example: "the cat" should have similar representations whether at positions (0,1) or (10,11)

💡 Add position information to attention scores

Instead of adding positional embeddings to inputs, bias the attention directly

$$\alpha_{ij}^h = \mathbf{Q}_h \mathbf{x}_i \cdot \mathbf{K}_h \mathbf{y}_j + b_{i-j}$$

where b_{i-j} depends only on the relative distance $i - j$

- Used in T5, DeBERTa, and many recent models
- Can handle sequences longer than training length
- Often clips distances: $b_{\min(k, i-j)}$ for some maximum k

Advantages

- **Better generalization** to different sequence lengths
- **More flexible** - model learns what distances matter
- Can be combined with absolute positions if needed

RoPE: Rotary Position Embedding

📖 RoFormer: Enhanced Transformer with Rotary Position Embedding: 2022. <http://arxiv.org/abs/2104.09864>.

🔧 Encode relative positions through rotation

Apply rotation $\mathbf{R}_{\Theta, m}^d$ to query and key vectors based on their positions m

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

where $\theta_i = 10000^{-2i/d}$

- **Key property:** $\mathbf{q}_m^T \mathbf{k}_j$ depends only on relative position $m - n$
- Used in GPT-3, LLaMA, ...
- Excellent extrapolation to longer sequences

📖 *Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation: 2022. <http://arxiv.org/abs/2108.12409>.*

💡 Simple linear penalty based on distance

Add a bias to attention scores proportional to distance:

$$\text{softmax}(q_i K^T + m \cdot [0, -1, -2, \dots, -(i-1)])$$

where m is a head-specific slope set **before** training (e.g., $m_h = 2^{-8h/H}$)

$$\begin{array}{ccccc}
 q_1 \cdot k_1 & & & & \\
 q_2 \cdot k_1 & q_2 \cdot k_2 & & & \\
 q_3 \cdot k_1 & q_3 \cdot k_2 & q_3 \cdot k_3 & & \\
 q_4 \cdot k_1 & q_4 \cdot k_2 & q_4 \cdot k_3 & q_4 \cdot k_4 & \\
 q_5 \cdot k_1 & q_5 \cdot k_2 & q_5 \cdot k_3 & q_5 \cdot k_4 & q_5 \cdot k_5
 \end{array}
 +
 \begin{array}{ccccc}
 0 & & & & \\
 -1 & 0 & & & \\
 -2 & -1 & 0 & & \\
 -3 & -2 & -1 & 0 & \\
 -4 & -3 & -2 & -1 & 0
 \end{array}
 \cdot m$$

- **No positional embeddings needed** - only bias in attention
- Different slopes for different heads
- Very strong extrapolation capability

FFN ACTIVATION FUNCTIONS HAVE EVOLVED

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

- Devlin, J. et al. 2018. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.
- Vaswani, A. et al. 2017. *Attention Is All You Need*.

GELU (2016, Transformers)

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right]$$

SWiGLU (2020, MODERN LLMs)

📖 *GLU Variants Improve Transformer: 2020.* <http://arxiv.org/abs/2002.05202>.

Based on Gated Linear Units (GLU)

📖 *Language Modeling with Gated Convolutional Networks: 2017.* <http://arxiv.org/abs/1612.08083>.

The SwiGLU activation function is defined as

$$\text{SwiGLU}(\mathbf{x}) = \text{Swish}_{\beta}(\mathbf{T}\mathbf{x} + \mathbf{b}) \otimes (\mathbf{U}\mathbf{x} + \mathbf{c})$$

where $\text{Swish}(\mathbf{x}) = x \cdot \sigma(\beta x)$

Improved performance over GELU

Used in most modern architectures

TWO OPTIONS

1. **Post** = After adding each residual, normalize the outputs
2. **Pre** = Before computing the residual, normalize the outputs

► Post-normalization (Original Transformer)

Self-attention block:

$$\mathbf{x}_{1:L}^{(\ell \angle SA)} = \text{LayerNorm}(\mathbf{x}_{1:L}^{(\ell-1)} + MHA(\mathbf{x}_{1:L}^{(\ell-1)}, \mathbf{x}_{1:L}^{(\ell-1)}))$$

Feedforward block:

$$\mathbf{x}_{1:L}^{(\ell)} = \text{LayerNorm}(\mathbf{x}_{1:L}^{(\ell \angle SA)} + \text{FFN}(\mathbf{x}_{1:L}^{(\ell \angle SA)}))$$

Normalization applied *after* residual connection

► Pre-normalization (Modern LLMs)

Self-attention block:

$$\mathbf{x}_{1:L}^{(\ell_{\angle SA})} = \mathbf{x}_{1:L}^{(\ell-1)} + MHA(\text{LayerNorm}(\mathbf{x}_{1:L}^{(\ell-1)}), \text{LayerNorm}(\mathbf{x}_{1:L}^{(\ell-1)}))$$

Feedforward block:

$$\mathbf{x}_{1:L}^{(\ell)} = \mathbf{x}_{1:L}^{(\ell_{\angle SA})} + FFN(\text{LayerNorm}(\mathbf{x}_{1:L}^{(\ell_{\angle SA})}))$$

Normalization applied *before* the transformation

► Pre-normalization (Modern LLMs)

Self-attention block:

$$\mathbf{x}_{1:L}^{(\ell \angle SA)} = \mathbf{x}_{1:L}^{(\ell-1)} + MHA(\text{LayerNorm}(\mathbf{x}_{1:L}^{(\ell-1)}), \text{LayerNorm}(\mathbf{x}_{1:L}^{(\ell-1)}))$$

Feedforward block:

$$\mathbf{x}_{1:L}^{(\ell)} = \mathbf{x}_{1:L}^{(\ell \angle SA)} + \text{FFN}(\text{LayerNorm}(\mathbf{x}_{1:L}^{(\ell \angle SA)}))$$

Normalization applied *before* the transformation

Comparison

- **Post-norm:** Original Transformer architecture
 - Requires careful learning rate warmup
 - Can be unstable with deep networks
- **Pre-norm:** Used in GPT, BERT variants, modern LLMs
 - More stable training / Better gradient flow
 - Easier to train very deep networks (e.g., GPT-3 with 96 layers)

LayerNorm (Original)

Normalizes across all features using mean and variance:

$$LN(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where $\mu = \frac{1}{d} \sum_{i=1}^d x_i$ and $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$

- Centers and scales the activations
- Learnable parameters: γ (scale) and β (shift)

RMSNorm: Root Mean Square Normalization

 Zhang, B. and Sennrich, R. 2019. Root Mean Square Layer Normalization.

Simplifies by removing mean centering:

$$\text{RMSNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}$$

where $\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$

- **No mean centering** - only scale normalization
- Only learnable scale parameter γ (no shift β)
- **Faster**: 7-64% speedup over LayerNorm
- **Simpler**: Fewer operations

RMSNorm: Root Mean Square Normalization

 Zhang, B. and Sennrich, R. 2019. Root Mean Square Layer Normalization.

Simplifies by removing mean centering:

$$\text{RMSNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}$$

where $\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$

- **No mean centering** - only scale normalization
- Only learnable scale parameter γ (no shift β)
- **Faster**: 7-64% speedup over LayerNorm
- **Simpler**: Fewer operations

Why the shift?

- Empirically, re-centering (subtracting mean) provides little benefit
- RMS normalization alone is sufficient for stable training

Encoders are better suited for **understanding** tasks (classification, extraction) whereas decoders are better for **generation** tasks (text generation, translation)

Ettin paired encoder & decoder

 *Seq vs Seq: An Open Suite of Paired Encoders and Decoders: 2025. <http://arxiv.org/abs/2507.11412>.*

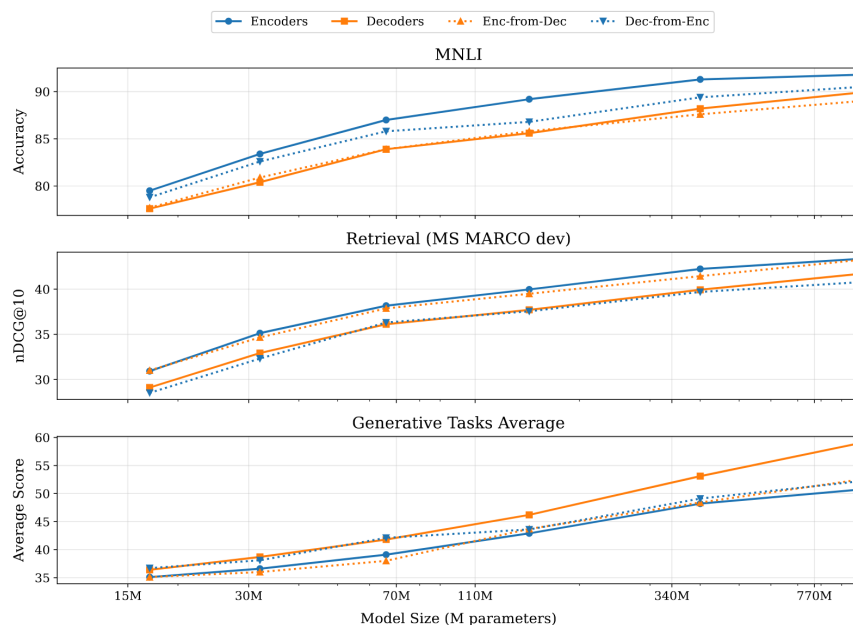
 Models with same sizes trained with the same data

Encoders are better suited for **understanding** tasks (classification, extraction) whereas decoders are better for **generation** tasks (text generation, translation)

Ettinger paired encoder & decoder

Seq vs Seq: An Open Suite of Paired Encoders and Decoders: 2025. <http://arxiv.org/abs/2507.11412>.

Models with same sizes trained with the same data



“ Models in the preferred architecture (e.g. encoders in MNLI) do better than the opposite architecture even with an order of magnitude greater size ”

TRANSFORMER ARCHITECTURE HAS EVOLVED SINCE 2017

- Positional embeddings: from absolute to relative (RoPE, ALiBi)
- Normalization: from LayerNorm post-norm to RMSNorm pre-norm
- Activation functions: from ReLU to SwiGLU
- Attention patterns: from dense to sparse / efficient (more on this later)

TRANSFORMER ARCHITECTURE HAS EVOLVED SINCE 2017

- Positional embeddings: from absolute to relative (RoPE, ALiBi)
- Normalization: from LayerNorm post-norm to RMSNorm pre-norm
- Activation functions: from ReLU to SwiGLU
- Attention patterns: from dense to sparse / efficient (more on this later)

BUT THE CORE PRINCIPLES REMAIN THE SAME!

- Self-Attention for contextualization (+ cross-attention in encoder-decoders)
- Feedforward networks for per-token computation
- Stacking layers with residual connections

TRANSFORMER ARCHITECTURE HAS EVOLVED SINCE 2017

- Positional embeddings: from absolute to relative (RoPE, ALiBi)
- Normalization: from LayerNorm post-norm to RMSNorm pre-norm
- Activation functions: from ReLU to SwiGLU
- Attention patterns: from dense to sparse / efficient (more on this later)

BUT THE CORE PRINCIPLES REMAIN THE SAME!

- Self-Attention for contextualization (+ cross-attention in encoder-decoders)
- Feedforward networks for per-token computation
- Stacking layers with residual connections

ENCODER VS DECODERS

- Encoders for understanding tasks
- Decoders for generation tasks
- Encoder-Decoders: multi-modal tasks (e.g., image captioning, table QA, ...)